

HaloFormCraft

Technical Documentation

Current Application & System Specification

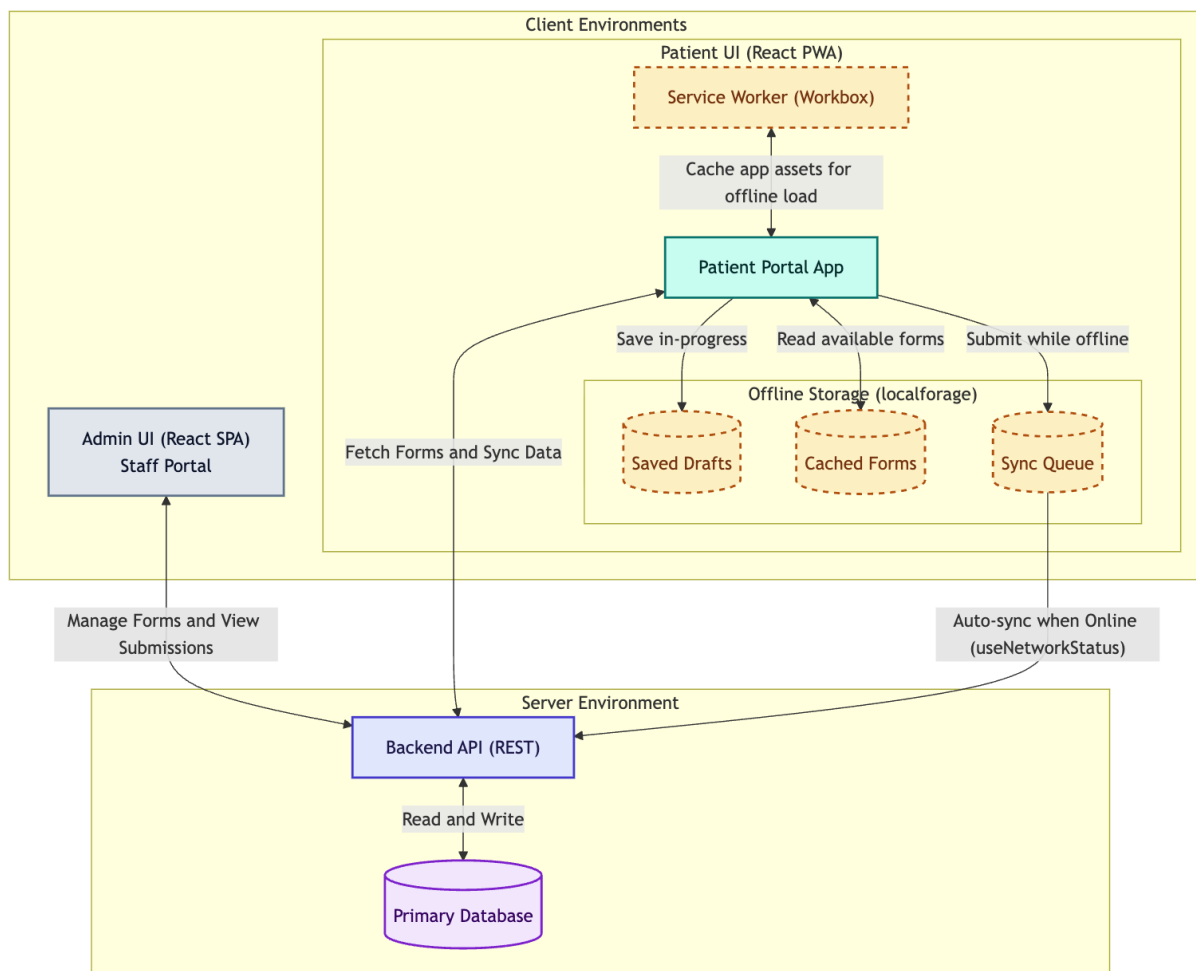
Generated for Internal Audit - May 2026

HaloFormCraft: High-Level Design (HLD) Document

This document outlines the system architecture, component design, data flow, and technical stack for the **HaloFormCraft** platform. The system is split into two primary front-end clients communicating with a central RESTful backend.

1. System Architecture Overview

The system follows a decoupled client-server architecture, utilizing a centralized backend API that serves two distinct frontend applications, each tailored for different user groups and environments.



2. Core Components

2.1 Admin UI (Staff Portal)

A standard React Single Page Application (SPA) designed for desktop use by hospital administrators and clinical staff with stable network connections.

Key Features:

- * **Form Builder:** A drag-and-drop interface allowing staff to design complex clinical forms dynamically.
- * **Project & Hospital Management:** Dashboards to organize forms by department or study

cohort. * **Response Viewer:** Interface to review, export, and analyze form submissions received from patients.

2.2 Patient UI (Patient Portal)

A Progressive Web App (PWA) designed for mobile devices and tablets, built with an **Offline-First** philosophy to handle unreliable internet connections in clinical environments.

Key Features: * **Advanced Input Types:** Supports specialized clinical inputs including QR Code scanning, dynamic calculations, geolocation capture, and digital signatures. * **Offline Capabilities:** Uses Service Workers to cache the application shell so it loads without an internet connection. * **Resilient Storage:** Utilizes local storage to safely store Form Templates, In-Progress Drafts, and Pending Submissions.

3. Data Flow

3.1 Form Creation Flow

1. **Admin User** logs into the **Admin UI**.
2. Using the FormBuilder, the admin designs a form template.
3. The template is sent via HTTP POST to the **Backend API** and stored in the **Primary Database**.

3.2 Form Consumption & Offline Synchronization Flow

The Patient UI utilizes a sophisticated queueing system to ensure zero data loss.

1. **Initial Load (Online):** The Patient UI fetches assigned form templates from the Backend API and caches them locally.
 2. **Form Entry (Online or Offline):** The patient fills out the form. They can exit the app and their progress is safely stored.
 3. **Form Submission:**
 4. **If Online:** The payload is sent directly to the Backend API.
 5. **If Offline:** The payload is written to the Sync Queue with a status of PENDING.
 6. **Reconnection & Auto-Sync:**
 7. The application listens for network reconnection.
 8. Upon reconnection, the sync service is triggered.
 9. The service iterates through the Sync Queue and attempts to POST pending items to the API.
-

4. Technical Stack

- **Frontend Framework:** React 19, Vite (Core UI framework and fast build tooling)
 - **Styling:** Tailwind CSS v4 (Utility-first CSS)
 - **State & Forms:** react-hook-form, yup (Performant form state management and schema validation)
 - **PWA & Caching:** vite-plugin-pwa, workbox (Service worker generation and caching)
 - **Local Storage:** localforage (Asynchronous storage wrapper for offline data persistence)
 - **Mapping & Location:** leaflet, react-leaflet (Interactive maps for GPS)
 - **Media & Hardware:** html5-qrcode, react-signature-canvas (Camera access and touch canvas)
-

Extensibility Note: The component-driven structure makes it highly modular. Adding a new clinical input type simply requires defining the React component in the Patient UI and adding its corresponding JSON schema representation to the Admin UI's Form Builder.